

Assignment no. 7

Exercise 1 Research: Context Window Management

LLMs have a limited context size, which limits the amount of content that can be fed into the model before the beginning of the context is lost (catastrophic forgetting). Even within the context size, the model will not treat all content equally. Take a look at the paper “Lost in the Middle: How Language Models Use Long Contexts” to gain better insight into how the positioning of content within the context matters.

Steps:

1. Read **chapter 2** (Multi-Document Question Answering) of the paper: [Lost in the Middle: How Language Models Use Long Contexts](#).
2. Summarize what you have learned and prepare to present your findings in class.
3. Research what can be done to mitigate this issue.

Deliverables:

- Your research on the “Lost in the Middle” problem and potential approaches to address it.

Exercise 2 LangChain Prompt & Chat Templates: Domain-Specific Q&A Assistant

Large Language Models can be adapted to specialized domains by carefully designing prompts and chat templates. In this exercise, you will build a reusable, domain-specific Q&A assistant using LangChain prompt templates instead of ad-hoc prompts.

The assistant should answer questions based on provided policy or guideline excerpts and produce answers in a consistent, structured format.

Tasks:

1. Design a LangChain [PromptTemplate](#) or chat prompt template that accepts at least the following inputs:
 - a domain document or policy excerpt,
 - a user question,
 - a parameter controlling the level of formality (e.g., casual, professional, legal).
2. Enforce a fixed response structure (e.g., short answer, justification, and reference to the provided text).
3. Test your template with multiple domain documents and questions.
4. Briefly discuss the advantages of prompt templates over inline prompting in larger applications.

Deliverables: A notebook containing the prompt template implementation, example interactions, and a short discussion of design decisions.

Exercise 3 LangChain Memory & Checkpointing: Long-Running Interview Assistant

Many applications require conversational state to persist over multiple interactions or even across program restarts. In this exercise, you will build a long-running interview or assessment bot that remembers past interactions using [LangChain memory](#) and [checkpointing](#).

The assistant should conduct an interview by asking one question at a time and adapt its behavior based on earlier answers.

Tasks:

1. Implement a conversational agent that asks a sequence of interview or assessment questions.
2. Use LangChain memory to store previous user answers and system decisions.
3. Add checkpointing so that the interview state can be saved and restored across multiple program runs.
4. Demonstrate that the interaction can be stopped and resumed without losing context.
5. Compare at least two different memory strategies (e.g., full buffer vs. summarized memory).

Deliverables: A notebook showing the interview flow, use of memory and checkpointing, and a short comparison of the memory approaches.

When using Large Language Models (e.g., ChatGPT, Copilot, ...) to produce output that is submitted, also state the prompt and model used in the submission. Submissions that are not marked as such and are noticeably AI-generated will be graded with zero points!

Submission: electronically via Moodle;
Deadline: Mon, May 4, 2026, 23:59.